CHMLTECH ONLINE JOURNAL · VOLUME 104, ISSUE 3 · RESEARCH ARTICLE

# Retrieval-Augmented Generation: Design, Evaluation and Failure Analysis for Enterprise Knowledge Systems

**Quak Foo Lee**
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Burnaby, BC, Canada

Published November 9, 2026  19 min read

ABSTRACT

Large language models hallucinate, have a fixed knowledge cutoff and have no access to an organization's private documents. Retrieval-augmented generation (RAG) addresses these limitations by conditioning generation on passages retrieved from an external corpus at inference time. This paper reviews the design, evaluation and failure analysis of RAG systems for enterprise knowledge applications from the perspective of November 2026. It presents the original RAG formulation, in which the output probability is marginalized over retrieved passages, together with the related REALM, Fusion-in-Decoder and RETRO architectures. It then examines the retrieval stage in detail: dense passage retrieval, BM25 sparse retrieval, hybrid search with reciprocal rank fusion, chunking strategies, embedding model selection and approximate nearest neighbor indexes such as HNSW. Re-ranking with cross-encoders and late-interaction models, query rewriting with hypothetical document embeddings, and advanced patterns including Self-RAG, Corrective RAG and GraphRAG are reviewed, along with evidence on long-context models versus retrieval. The paper defines retrieval metrics (recall@k, mean reciprocal rank and nDCG) and the RAGAS generation

metrics, maps the seven documented failure points of RAG systems to diagnostic signals and mitigations, and discusses indirect prompt injection, knowledge poisoning and data leakage. It concludes with an implementation checklist and argues that RAG systems should be engineered for diagnosis, with retrieval and generation measured separately and retrieved content treated as untrusted input.

**Keywords:** retrieval-augmented generation; large language models; dense retrieval; hybrid search; re-ranking; RAG evaluation; prompt injection; enterprise knowledge systems

## 1. Introduction

Large language models (LLMs) store a great deal of factual and procedural knowledge in their parameters, but this parametric memory has three properties that limit its use in enterprise knowledge systems. First, models produce fluent statements that are unsupported by, or contradict, the available evidence, a behavior generally described as hallucination [1]. Second, parametric knowledge is fixed at the end of training, so a model cannot know about documents, prices, policies or events that postdate its training corpus without being retrained or fine-tuned. Third, the documents that matter most to an organization, such as engineering standards, contracts, procedures, support tickets and internal reports, are usually private and therefore absent from any public pretraining corpus.

Retrieval-augmented generation (RAG) addresses all three limitations by coupling a generator with a non-parametric memory: at inference time a retriever selects passages from an external corpus, and the generator conditions its output on those passages [2, 3]. The external corpus can be updated without retraining, access to it can be governed with the same controls that protect the source documents, and the retrieved passages provide evidence that can be cited and audited. Gao et al. characterize the field's progression from "naive" retrieve-then-read pipelines, through "advanced" pipelines with pre- and post-retrieval optimization, to "modular" architectures in which retrieval, routing, rewriting and verification are composable components [3].

RAG does not, however, remove the need for engineering discipline. A RAG system inherits the failure modes of information retrieval (relevant content that is missing or ranked too low), the failure modes of LLMs (ignoring or misreading evidence), and new failure modes that arise

from their interaction, including attacks delivered through the retrieved content itself. This paper reviews the design space of RAG from the perspective of November 2026 with an emphasis on enterprise knowledge systems. Section 2 presents the original formulations and their predecessors and successors. Section 3 covers retrieval: sparse, dense and hybrid methods, chunking, embedding models and vector indexes. Section 4 covers re-ranking and query transformation. Section 5 reviews advanced patterns and the relationship between RAG and long-context models. Section 6 treats evaluation, Section 7 failure analysis, Section 8 security and privacy, and Section 9 presents an implementation checklist. Section 10 concludes. Figure 1 maps the stages discussed in these sections onto a single pipeline and marks where each failure point of Section 7 typically originates, so that a failed answer can be traced to the stage where the evidence was lost.

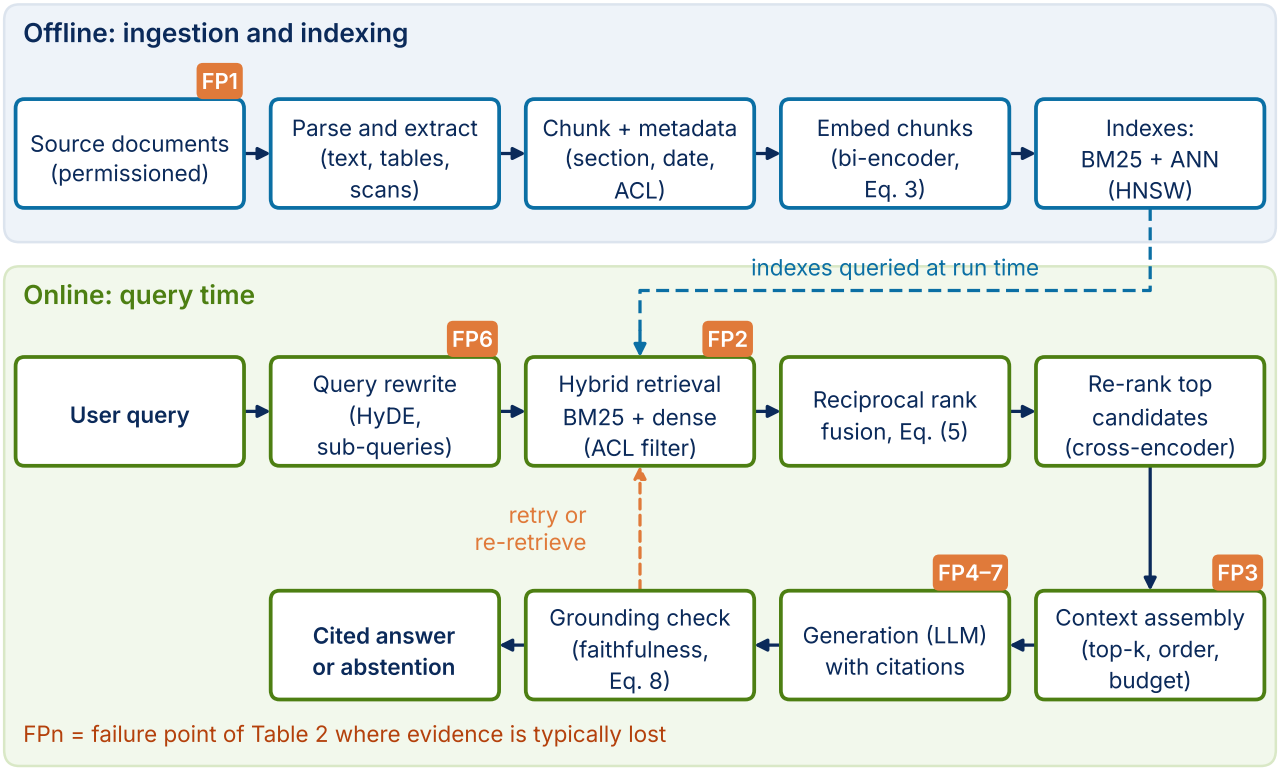



**Figure 1.** End-to-end enterprise RAG pipeline. Offline, documents are parsed, chunked with metadata, embedded and indexed; at query time the query is rewritten, retrieved from sparse and dense indexes, fused with RRF, re-ranked, assembled into a prompt, answered and checked for grounding. Orange tags mark the failure points FP1–FP7 of Table 2 at the stage where each typically originates.

Drawn by the author from Sections 3–7; failure points as listed in Table 2.

## 2. Foundations of Retrieval-Augmented Generation

## 2.1 The Original RAG Formulation

Lewis et al. introduced the term retrieval-augmented generation for models that combine a pretrained sequence-to-sequence generator, the parametric memory, with a dense vector index of Wikipedia, the non-parametric memory, accessed through a pretrained neural retriever [2]. Given an input sequence $x$, a retriever $p_\eta(z \mid x)$ returns a distribution over passages $z$, and a generator $p_\theta(y_i \mid x, z, y_{1:i-1})$ produces the output token by token. Because the retrieved passage is a latent variable, the output probability is obtained by marginalizing over the top-$k$ retrieved passages. In the RAG-Sequence variant the same passage is used for the whole output sequence:

$$p_{\text{RAG-Seq}}(y \mid x) \approx \Sigma_{z \in \text{top-}k(p_\eta(\cdot \mid x))} \, p_\eta(z \mid x) \, \Pi_{i=1}^{N} \, p_\theta(y_i \mid x, z, y_{1:i-1}) \tag{1}$$

In the RAG-Token variant the marginalization is performed separately for each output token, so that different tokens can draw on different passages:

$$p_{\text{RAG-Tok}}(y \mid x) \approx \Pi_{i=1}^{N} \, \Sigma_{z \in \text{top-}k(p_\eta(\cdot \mid x))} \, p_\eta(z \mid x) \, p_\theta(y_i \mid x, z, y_{1:i-1}) \tag{2}$$

The retriever and generator were fine-tuned jointly on input–output pairs, with the document encoder and index held fixed, and the resulting models set new results on several open-domain question-answering benchmarks while producing more specific and factual text than a parametric-only baseline [2]. Two features of Eqs. (1) and (2) are important for practitioners. The retrieval score enters the output probability directly, so a poorly calibrated retriever degrades generation; and the truncation to the top $k$ passages means that any relevant passage ranked below $k$ is invisible to the generator.

## 2.2 REALM, Fusion-in-Decoder and RETRO

Retrieval-augmented language modeling predates the RAG name. REALM augmented masked-language-model pretraining with a learned knowledge retriever and backpropagated through the retrieval step, treating the retrieved document as a latent variable in the same marginal form, $p(y \mid x) = \Sigma_z \, p(y \mid z, x) \, p(z \mid x)$, and periodically refreshing the document index during training [4]. Fusion-in-Decoder (FiD) took a different route: each retrieved passage is encoded independently together with the question, and the decoder attends over the concatenation of

all encoded passages, which allows a generator to aggregate evidence from many passages efficiently [5].

RETRO (Retrieval-Enhanced Transformer) moved retrieval into the language model itself. It conditions an autoregressive model on document chunks retrieved from a database of 2 trillion tokens on the basis of local similarity with the preceding tokens, using a frozen BERT retriever and a chunked cross-attention mechanism; with this database it obtained performance comparable to GPT-3 and Jurassic-1 on the Pile while using 25× fewer parameters [6]. RETRO demonstrated that retrieval can substitute for a large share of parametric capacity, but most enterprise systems today follow the simpler architecture of Lewis et al., in which an off-the-shelf instruction-tuned LLM receives retrieved passages in its prompt without any joint training [3].

# 3. Retrieval: Sparse, Dense and Hybrid

## 3.1 Dense Retrieval

Dense passage retrieval (DPR) encodes questions and passages with two separate BERT-based encoders, $E_{\mathrm{Q}}$ and $E_{\mathrm{P}}$, and scores a passage by the inner product of the two embeddings [7]:

$$\operatorname{sim}(q, p) = E_{\mathrm{Q}}(q)^{\mathrm{T}}\, E_{\mathrm{P}}(p) \tag{3}$$

The encoders are trained contrastively so that a question's embedding is closer to its positive passage than to negative passages, including in-batch negatives and hard negatives mined with BM25. Because passage embeddings are computed offline, retrieval at query time reduces to a maximum inner product search. Karpukhin et al. reported that this dual-encoder retriever outperformed a strong Lucene-BM25 system by 9–19 percentage points absolute in top-20 passage retrieval accuracy across a range of open-domain question-answering datasets [7]. Dense retrieval captures paraphrase and semantic similarity that exact term matching misses, which is why it became the default first stage in RAG pipelines.

## 3.2 Sparse Retrieval and BM25

Sparse retrieval represents queries and documents as weighted bags of terms. The dominant scoring function, Okapi BM25, derives from the probabilistic relevance framework [8]. For a query $Q$ containing terms $t$ and a document $D$:

$$\mathrm{BM25}(D, Q) = \Sigma_{t \in Q} \, \mathrm{IDF}(t) \cdot [f(t, D) \, (k_1 + 1)] \, / \, [f(t, D) + k_1 \, (1 - b + b \cdot |D| \, / \, \mathrm{avgdl})$$

Here $f(t, D)$ is the frequency of term $t$ in $D$, $|D|$ is the document length, avgdl is the average document length in the collection, $N$ is the number of documents and $n(t)$ is the number of documents containing $t$. The parameter $k_1$ controls how quickly term-frequency saturates and $b$ controls length normalization [8]. Implementations differ in their IDF variant; several add a constant inside the logarithm to keep weights non-negative for very common terms. BM25 remains important in enterprise RAG for two reasons. It matches exact identifiers such as part numbers, error codes, clause references and personal names, which dense encoders often blur; and it requires no training, so it transfers to new domains reliably. The BEIR benchmark showed that BM25 is a robust zero-shot baseline across heterogeneous retrieval tasks and that many neural retrievers trained on one dataset generalize poorly to others [9].

## 3.3 Hybrid Search and Reciprocal Rank Fusion

Hybrid search runs sparse and dense retrievers in parallel and merges their ranked lists. Score-level fusion requires normalizing scores that have different scales and distributions, so rank-level fusion is often preferred. Reciprocal rank fusion (RRF) scores each document $d$ by summing reciprocal ranks over the set of rankings $R$ [10]:

$$\mathrm{RRF}(d) = \Sigma_{r \in R} \, 1 \, / \, (k + r(d)) \tag{5}$$

where $r(d)$ is the rank of $d$ in ranking $r$. The constant $k$ mitigates the influence of high rankings by outlier systems; Cormack et al. fixed $k = 60$ in a pilot study and found that RRF outperformed Condorcet fusion and individual learning-to-rank methods on TREC collections [10]. RRF needs no score calibration and no training data, which makes it a practical default for combining BM25 with one or more dense retrievers. Table 1 summarizes the main retrieval families. Figure 2 works through Eq. (5) for two short ranked lists: a passage found by both retrievers at moderate ranks outscores a passage ranked first by only one of them.

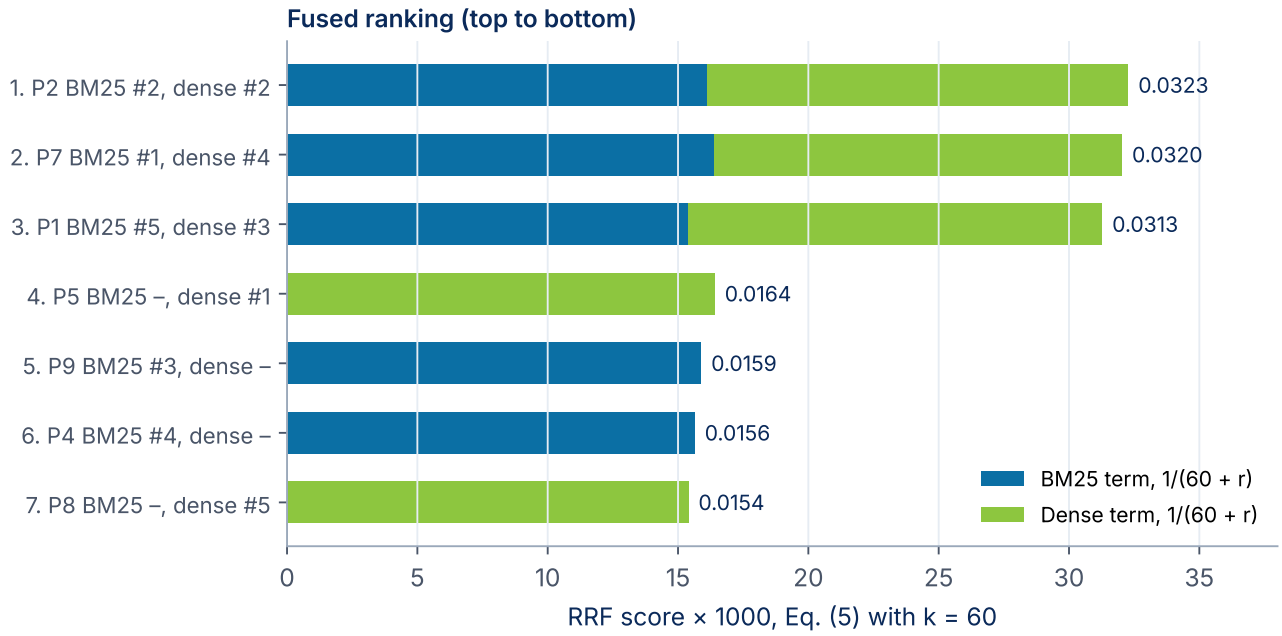



**Figure 2.** Worked example of reciprocal rank fusion, Eq. (5), with k = 60, for hypothetical BM25 and dense top-5 lists of passages P1–P9. Each bar is split into the contribution of each retriever; the labels give the passage's rank in each list.

Author's calculation from Eq. (5) for hypothetical rankings, with k = 60 as in [10].

**Table 1.** Principal retrieval families used in RAG pipelines.

| Family | Representation and scoring | Strengths | Weaknesses |
|---|---|---|---|
| Sparse (BM25) | Term-weighted bag of words; Eq. (4) | Exact matching of identifiers and rare terms; no training; robust zero-shot [8, 9] | Vocabulary mismatch; no notion of paraphrase |
| Dense bi-encoder (DPR, sentence embeddings) | Single vector per text; inner product or cosine, Eq. (3) | Semantic matching; fast approximate search over precomputed vectors [7, 11] | Domain shift; weaker on exact identifiers; needs an embedding model and index |
| Late interaction (ColBERT) | One vector per token; sum of maximum similarities | Finer-grained matching than single-vector models at moderate cost [12] | Larger index; more complex serving |
| Cross-encoder re-ranker | Query and passage encoded jointly; relevance score | Highest precision among these options [13] | Too slow for first-stage retrieval over a large corpus |
| Hybrid with RRF | Rank fusion of several retrievers; Eq. (5) | Combines lexical and semantic evidence without score calibration [10] | Extra latency and infrastructure for multiple indexes |

Source: compiled from [8], [7], [12], [13] and [10].

## 3.4 Chunking Strategies

Enterprise documents are usually too long to embed or place in a prompt in their entirety, so they are split into chunks before indexing. The chunk is simultaneously the unit of retrieval, the unit of embedding and the unit of evidence presented to the generator, and these roles pull in different directions: small chunks yield focused embeddings and precise citations but can separate a statement from the context that qualifies it, while large chunks preserve context but dilute the embedding and consume prompt budget [3]. Common strategies include fixed-length windows with overlap, splitting on document structure (headings,

sections, table boundaries), recursive splitting that falls back from paragraphs to sentences, and semantic splitting at points where adjacent-sentence similarity drops. Gao et al. also describe small-to-big patterns, in which small units are retrieved but their enclosing section is passed to the generator, and the enrichment of chunks with metadata such as title, section path, date and access-control labels that can be used for filtering [3].

No chunking strategy is optimal across corpora, so chunking should be treated as a tunable parameter and evaluated with the retrieval metrics of Section 6. Tables, forms and scanned drawings require dedicated handling, because text extraction that flattens a table into a stream of cells destroys the row–column relationships on which the answer depends.

## 3.5 Embedding Models and Vector Indexes

Sentence-BERT established the now standard approach of fine-tuning a Transformer encoder in a Siamese configuration so that cosine similarity between pooled embeddings reflects semantic similarity, reducing the cost of pairwise similarity search from cross-encoding every pair to a single encoding per sentence [11]. The Massive Text Embedding Benchmark (MTEB) evaluates embedding models across retrieval, classification, clustering, re-ranking, semantic similarity and other tasks, and found that no single model dominated across all tasks [14]. Public leaderboard rank is therefore only a starting point: an embedding model should be selected by measuring retrieval quality on a sample of the organization's own queries and documents, with attention to language coverage, maximum input length, embedding dimension (which drives index size) and whether the model can be hosted within the organization's data-residency constraints.

Exact nearest neighbor search scales linearly with corpus size, so production systems use approximate nearest neighbor (ANN) indexes. Hierarchical Navigable Small World (HNSW) graphs organize vectors into a multi-layer proximity graph in which upper layers contain progressively sparser subsets of points; a search starts at the top layer and greedily descends, giving logarithmic complexity scaling in the authors' evaluation and strong recall–speed trade-offs [15]. The FAISS library provides GPU implementations of inverted-file and product-quantization indexes that compress vectors and make billion-scale search feasible [16]. ANN parameters, such as the HNSW graph degree and search breadth, trade recall for latency and memory; because a missed neighbor at this stage cannot be recovered by later stages, index recall should be measured against exact search on a held-out query sample.

# 4. Re-ranking and Query Transformation

## 4.1 Cross-Encoders and Late Interaction

First-stage retrievers are optimized for recall at low cost. A second-stage re-ranker then reorders a candidate set of, typically, tens to hundreds of passages using a more expensive model. Nogueira and Cho showed that a BERT cross-encoder, which concatenates query and passage and predicts relevance from the joint representation, substantially improved passage ranking on the MS MARCO benchmark over prior methods [13]. Because the cross-encoder must process every query–passage pair, it is applied only to the candidates returned by the first stage.

ColBERT occupies a middle ground. It encodes queries and documents independently into one embedding per token and computes relevance by late interaction: for each query token it takes the maximum similarity over document tokens and sums these maxima. Document token embeddings can be precomputed, so ColBERT retains much of the expressiveness of token-level interaction while remaining far cheaper than a cross-encoder at query time [12]. In practice a two- or three-stage cascade, hybrid retrieval followed by a learned re-ranker, is the most common way to raise precision in the top few positions that the generator actually reads.

## 4.2 Query Rewriting and HyDE

User questions are often short, ambiguous or phrased differently from the documents that answer them. Query transformation techniques rewrite the query before retrieval: expanding acronyms, decomposing multi-part questions into sub-queries, generating several paraphrases whose results are fused with Eq. (5), or adding conversational context in multi-turn settings [3]. Hypothetical Document Embeddings (HyDE) take this further. An instruction-following LLM is asked to write a hypothetical document that answers the query; this document may contain false details, but it captures the relevance pattern. It is then encoded with an unsupervised contrastive encoder, and real documents near its embedding are retrieved, so that the encoder's dense bottleneck grounds the hypothetical text in the actual corpus [17]. Gao et al. reported that HyDE significantly outperformed the unsupervised Contriever retriever and was comparable to fine-tuned retrievers across web search, question answering and fact verification tasks [17]. Query rewriting adds an LLM call to the critical path, so its latency and cost must be justified by measured retrieval gains.

The generator can only use what the retriever surfaces in the top *k* positions. Investment in hybrid retrieval, re-ranking and chunking usually yields larger and more measurable gains than changes to the prompt or the generator model.

# 5. Advanced Patterns and Long Context

## 5.1 Self-RAG and Corrective RAG

A fixed pipeline retrieves a fixed number of passages for every query, whether or not retrieval is needed and whether or not the passages are relevant. Self-RAG trains a single LM to retrieve on demand and to generate special reflection tokens that critique whether retrieval is needed, whether each retrieved passage is relevant, whether the generated segment is supported by it, and how useful the response is. Because reflection tokens are explicit, behavior can be tuned at inference time; Asai et al. reported that Self-RAG models with 7 and 13 billion parameters outperformed ChatGPT and retrieval-augmented Llama 2-chat on open-domain question answering, reasoning and fact verification, with gains in factuality and citation accuracy for long-form generation [18].

Corrective RAG (CRAG) addresses the case in which retrieval goes wrong. A lightweight retrieval evaluator assigns a confidence to the retrieved set for a query; depending on this confidence, the system uses the documents, discards them and falls back to web search, or combines both. A decompose-then-recompose step filters retrieved documents down to their relevant parts [19]. CRAG is designed as a plug-in to existing pipelines. For enterprise use, the important idea is the explicit retrieval-quality gate; the web-search fallback is frequently inappropriate for confidential questions and is better replaced by an abstention or escalation path.

## 5.2 GraphRAG

Conventional RAG retrieves passages that are locally similar to the query. It therefore performs poorly on global sensemaking questions directed at an entire corpus, such as "What are the main themes in this dataset?", which are query-focused summarization tasks rather than retrieval tasks. GraphRAG uses an LLM to build a graph index in two stages: it extracts an entity knowledge graph from the source documents and then pregenerates summaries for communities of closely related entities. At query time, each community summary contributes a

partial answer, and the partial answers are summarized into a final response [20]. For global questions over datasets in the one-million-token range, Edge et al. reported substantial improvements over a conventional RAG baseline in the comprehensiveness and diversity of answers [20]. The approach has a significant indexing cost, because the LLM must read the entire corpus, so it is best reserved for corpora and question types where global synthesis is a stated requirement.

### 5.3 Long Context Versus Retrieval

As context windows have grown, a recurring question is whether retrieval is still needed when an entire document collection can be placed in the prompt. Two lines of evidence suggest that it is. Liu et al. found that language models' performance on multi-document question answering and key-value retrieval was often highest when relevant information was at the beginning or end of the input and degraded significantly when it was in the middle, even for models designed for long contexts [21]. Xu et al. compared retrieval augmentation with context-window extension using a 43-billion-parameter GPT model and Llama 2-70B; an LLM with a 4K window and simple retrieval achieved performance comparable to a model fine-tuned for a 16K window on long-context tasks with much less computation, and retrieval improved performance regardless of the extended window size [22].

For enterprise systems there are additional, non-accuracy reasons to retrieve. Placing a large corpus in every prompt raises cost and latency per query, makes it difficult to enforce document-level access controls, and obscures which source supported an answer. A reasonable design position is to use long context to pass more complete retrieved units (whole sections rather than fragments) and to order the most relevant evidence at the start or end of the prompt, rather than to abandon retrieval [21, 22].

## 6. Evaluation

### 6.1 Retrieval Metrics

Evaluation should separate the retrieval stage from the generation stage, because a single end-to-end score cannot distinguish a missing document from a misread one. Retrieval is evaluated against a set of queries $Q$ with known relevant passages. Recall at cutoff $k$ measures the fraction of relevant items for query $q$, $\mathrm{Rel}(q)$, that appear among the top $k$

results, $\mathrm{Ret}_k(q)$; mean reciprocal rank (MRR) averages the reciprocal of the rank of the first relevant result:

$$\mathrm{Recall@}k = (1/|Q|)\,\Sigma_{q\in Q}\,|\mathrm{Rel}(q)\cap \mathrm{Ret}_k(q)|\,/\,|\mathrm{Rel}(q)|,\ \mathrm{MRR} = (1/|Q|)\,\Sigma_{q\in Q}\,1/\mathrm{rank}$$

Graded relevance judgments are handled by discounted cumulative gain, which rewards relevant results more when they appear higher in the ranking [23]. With graded relevance $rel_i$ at position $i$, a widely used form is:

$$\mathrm{DCG@}k = \Sigma_{i=1}^{k}\,(2^{rel_i}-1)\,/\,\log_2(i+1),\ \mathrm{nDCG@}k = \mathrm{DCG@}k\,/\,\mathrm{IDCG@}k \tag{7}$$

where IDCG@$k$ is the DCG of the ideal ordering, so that nDCG lies between 0 and 1. Järvelin and Kekäläinen's original formulation used a logarithmic discount applied after a chosen rank; the exponential-gain variant in Eq. (7) is common in modern benchmarks and emphasizes highly relevant documents [23, 9]. For RAG, recall@$k$ at the value of $k$ actually passed to the generator is the most directly interpretable metric, because it bounds the information available for generation, while MRR and nDCG indicate whether the best evidence appears early enough to avoid the position effects of Section 5.3. Figure 3 illustrates the difference for one query: re-ranking the same ten candidates leaves recall@10 unchanged but raises recall@5 from 0.67 to 1.00 and nDCG@5 from 0.43 to 0.98.

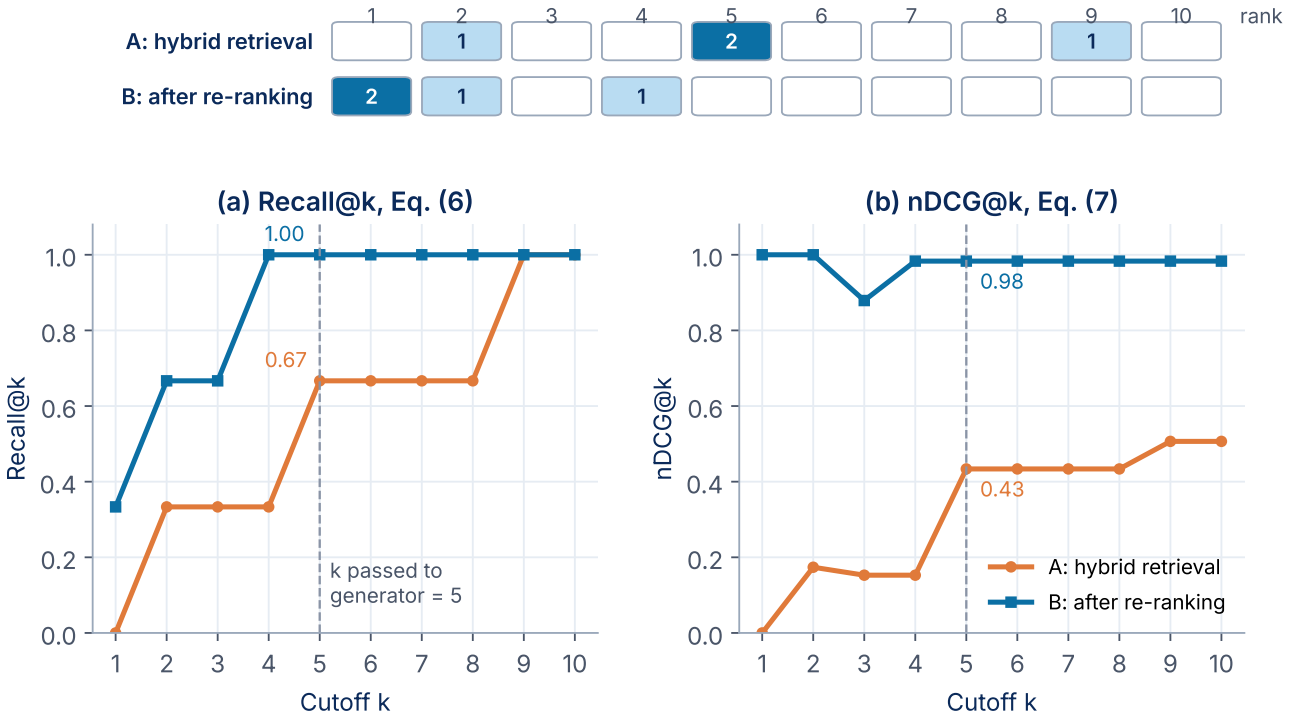



**Figure 3.** Recall@k and nDCG@k from Eqs. (6) and (7) for one hypothetical query with three relevant passages (graded 2, 1 and 1). Ordering A is a first-stage hybrid list; ordering B is the same ten candidates after re-ranking. The dashed line marks the five passages passed to the generator.

Author's calculation from Eqs. (6) and (7) for hypothetical relevance judgments.

## 6.2 Generation Metrics and RAGAS

Generation quality has several dimensions that must be measured separately: whether the answer is supported by the retrieved context, whether it addresses the question, and whether the context itself was relevant and focused. RAGAS proposes reference-free metrics for these three aspects, named faithfulness, answer relevance and context relevance, computed with an LLM so that they do not require human-written ground-truth answers [24]. Faithfulness is computed by having an LLM decompose the answer into a set of statements $S$ and then judge, for each statement, whether it can be inferred from the context; with $V$ the set of supported statements:

$$F = |V| / |S| \tag{8}$$

Answer relevance is computed by generating candidate questions from the answer and averaging their embedding similarity with the original question [24]. LLM-based judges are convenient for rapid iteration, but they are themselves models with errors and biases, so their agreement with human judgments should be measured on a sample of the organization's own data before their scores are used as release criteria. A practical evaluation set combines a few hundred representative questions with labeled relevant passages and reference answers, a smaller adversarial set (unanswerable questions, conflicting sources, questions that require tables), and production logs sampled for periodic human review.

## 7. Failure Analysis

Barnett et al. reported an experience study of three RAG systems in research, education and biomedical domains and identified seven failure points (FP) [25]. Two of their conclusions are particularly relevant to enterprise programs: validation of a RAG system is only feasible during operation, and robustness evolves through operation rather than being designed in at the start [25]. Table 2 lists the failure points, the pipeline stage where each originates, the metrics from Section 6 that expose it, and common mitigations.

**Table 2.** Failure points of RAG systems mapped to pipeline stages, diagnostic signals and mitigations.

| Failure point | Stage | Diagnostic signal | Typical mitigations |
|---|---|---|---|
| FP1 Missing content: the question cannot be answered from the corpus, but the system answers anyway | Corpus and ingestion | Unanswerable test questions answered with confidence | Coverage audits; explicit abstention instructions; retrieval-confidence gate [19] |
| FP2 Missed the top-ranked documents: the answer exists but ranks below the cutoff | Retrieval | Low recall@$k$ (Eq. 6) | Hybrid search with RRF; better embeddings; tuning $k$ and ANN recall [10, 15] |
| FP3 Not in context: retrieved documents with the answer are dropped during consolidation | Context assembly | Relevant passage retrieved but absent from the final prompt | Re-ranking before truncation; budget-aware packing; small-to-big expansion [13, 3] |
| FP4 Not extracted: the answer is in the context but the LLM fails to extract it | Generation | Low answer correctness with high context recall | Fewer, cleaner passages; evidence placed at prompt start or end; conflict detection [21] |
| FP5 Wrong format: the requested format (for example, a table or list) is ignored | Generation | Schema or format validation failures | Structured-output constraints; output validators with retry |
| FP6 Incorrect specificity: the answer is too general or too specific | Query understanding and generation | Human review; answer-relevance scores [24] | Query rewriting and clarification; user-role context [17] |
| FP7 Incomplete: available information is omitted | Generation | Partial matches against reference answers | Query decomposition; multi-passage aggregation [5] |

Source: failure points from [25]; diagnostic and mitigation mapping by the author, drawing on the cited works.

Several failure points compound. A chunking choice that separates a table from its header (Section 3.4) produces FP2 at retrieval and FP4 at generation; a retriever that returns many near-duplicate passages pushes the decisive passage out of the prompt and produces FP3. Diagnosis therefore benefits from logging every intermediate artifact for each query: the rewritten query, the candidates from each retriever with their scores, the re-ranked list, the final prompt and the response. With these traces, a failed answer can be attributed to the earliest stage at which the relevant evidence was lost.

> Measure retrieval and generation separately. If the relevant passage is not in the prompt, the fault lies upstream of the generator, and prompt changes will not repair it; if it is in the prompt but the answer is wrong, retrieval changes will not repair it.

# 8. Security and Privacy

## 8.1 Indirect Prompt Injection and Knowledge Poisoning

A RAG system executes a language model over text that it did not author. Greshake et al. demonstrated that adversaries can place instructions in content that an LLM-integrated application later retrieves, such as web pages, e-mails or documents, and thereby manipulate the application's behavior without direct access to the prompt; their taxonomy of impacts includes data theft, worming and information ecosystem contamination, and they showed that processing retrieved prompts can manipulate an application's functionality and control whether and how other APIs are called [26]. Retrieved content must therefore be treated as untrusted data, never as instructions, and this principle must be enforced architecturally: tool permissions available to a RAG agent should be limited to what the use case needs, and actions with external effects should require confirmation.

Knowledge poisoning targets the corpus rather than the prompt. PoisonedRAG formulates the injection of malicious texts into a knowledge database as an optimization problem, so that a chosen question retrieves the attacker's text and the LLM produces an attacker-chosen answer; Zou et al. reported a 90% attack success rate when injecting five malicious texts per

target question into a knowledge database containing millions of texts, and found the defenses they evaluated to be insufficient [27]. In an enterprise setting the most effective controls are upstream: restricting who can write to indexed sources, recording provenance for every chunk, and monitoring for newly ingested content that is unusually similar to high-value queries.

### 8.2 Data Leakage and Access Control

RAG also creates privacy risk for the retrieval corpus. Zeng et al. showed empirically, using novel attack methods, that RAG systems are vulnerable to leaking their private retrieval database; they also found that retrieval can reduce leakage of the LLM's own training data [28]. The most common enterprise leakage path, however, is simpler: a shared index that ignores the permissions of the source systems, so that a user can retrieve passages from documents they could not open directly. Access control should be enforced at retrieval time by filtering on permission metadata attached to each chunk, with permissions synchronized from the source system, rather than by instructing the model not to reveal certain content.

> Treat everything retrieved as untrusted input. Enforce document permissions in the retriever, keep write access to indexed sources narrow, and give the model no capability whose misuse an injected instruction could exploit.

## 9. Enterprise Implementation Checklist

Table 3 consolidates the preceding sections into a checklist organized by pipeline stage. It is intended as a review aid for design and go-live decisions rather than a prescription; the appropriate depth at each stage depends on the risk of the use case.

**Table 3.** Implementation checklist for enterprise RAG systems.

| Stage | Design questions | Evidence to require before go-live |
|---|---|---|
| Scope and corpus | Which questions must be answered? Which sources are authoritative, and how current must they be? | Documented question taxonomy; source inventory with owners and refresh frequency; set of unanswerable questions (FP1) [25] |
| Ingestion and chunking | How are tables, scans and structured fields extracted? What chunk size, overlap and metadata? | Extraction spot checks; chunking variants compared on recall@$k$ [3] |
| Retrieval | Sparse, dense or hybrid? Which embedding model? What ANN settings? | Recall@$k$, MRR and nDCG on in-domain queries; ANN recall against exact search [14, 15, 10] |
| Re-ranking and query handling | Is a cross-encoder or late-interaction re-ranker justified? Is query rewriting needed? | Measured precision gain versus added latency [13, 12, 17] |
| Context assembly and generation | How many passages, in what order? How are citations and abstentions produced? | Faithfulness and answer relevance on the test set; citation accuracy audit [24, 21] |
| Security and privacy | Are source permissions enforced at retrieval? Who can write to indexed sources? Which tools can the model call? | Permission-filter tests; prompt-injection and poisoning red-team results [26, 27, 28] |
| Operations | What is logged per query? How are regressions detected after corpus or model updates? | Trace logging of intermediate artifacts; scheduled re-evaluation; human review of sampled production answers [25] |

Source: synthesis of Sections 3–8; see the cited works for each item.

Two organizational points complement the checklist. First, each authoritative source needs a named owner, because answer quality is bounded by source quality and no retrieval technique compensates for an outdated procedure. Second, evaluation sets should be versioned with the system, so that every change to chunking, embedding model, re-ranker, prompt or generator can be compared against the same baseline before it is released [25].

## 10. Conclusion

Retrieval-augmented generation grounds language-model outputs in an external, updatable and governable corpus. Its probabilistic foundation, the marginalization over retrieved passages in Eqs. (1) and (2), makes explicit that generation quality is bounded by retrieval quality, and the empirical record from DPR, BM25 benchmarks, reciprocal rank fusion and cross-encoder re-ranking shows that robust retrieval in enterprise settings usually combines lexical and semantic evidence in a multi-stage cascade [2, 7, 9, 10, 13]. Advanced patterns such as Self-RAG, Corrective RAG and GraphRAG add adaptive retrieval, quality gating and corpus-level synthesis, at the cost of additional training or indexing effort [18, 19, 20]. Longer context windows complement rather than replace retrieval, given position effects and the cost, access-control and provenance requirements of enterprise use [21, 22].

The engineering lesson of the literature is that RAG systems should be built to be diagnosed. Retrieval and generation metrics should be measured separately, intermediate artifacts should be logged, failure points should be traced to the stage at which evidence was lost, and retrieved content should be treated as untrusted input [24, 25, 26]. Organizations that adopt these practices obtain systems whose answers can be checked against sources and whose failures can be located and corrected.

## References


1. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1–38. 10.1145/3571730
2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33 (NeurIPS 2020). arxiv.org/abs/2005.11401
3. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997. arxiv.org/abs/2312.10997
4. Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). Retrieval augmented language model pre-training. Proceedings of the 37th International Conference on Machine Learning, PMLR 119, 3929–3938. proceedings.mlr.press/v119/guu20a.html
5. Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. Proceedings of the 16th Conference of the European

Chapter of the Association for Computational Linguistics: Main Volume, 874–880. 10.18653/v1/2021.eacl-main.74

6. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., de Las Casas, D., Guy, A., et al. (2022). Improving language models by retrieving from trillions of tokens. Proceedings of the 39th International Conference on Machine Learning, PMLR 162, 2206–2240. proceedings.mlr.press/v162/borgeaud22a.html
7. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 6769–6781. 10.18653/v1/2020.emnlp-main.550
8. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 4(1-2), 1–174. 10.1561/1500000019
9. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track. arXiv:2104.08663. arxiv.org/abs/2104.08663
10. Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759. 10.1145/1571941.1572114
11. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 3980–3990. 10.18653/v1/D19-1410
12. Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 39–48. 10.1145/3397271.3401075
13. Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085. arxiv.org/abs/1901.04085
14. Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, 2014–2037. 10.18653/v1/2023.eacl-main.148
15. Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern

Analysis and Machine Intelligence, 42(4), 824–836. 10.1109/TPAMI.2018.2889473

16. Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547. 10.1109/TBDATA.2019.2921572
17. Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise zero-shot dense retrieval without relevance labels. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 1762–1777. 10.18653/v1/2023.acl-long.99
18. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. The Twelfth International Conference on Learning Representations (ICLR 2024). arxiv.org/abs/2310.11511
19. Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884. arxiv.org/abs/2401.15884
20. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130. arxiv.org/abs/2404.16130
21. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12, 157–173. 10.1162/tacl_a_00638
22. Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoeybi, M., & Catanzaro, B. (2024). Retrieval meets long context large language models. The Twelfth International Conference on Learning Representations (ICLR 2024). arXiv:2310.03025. arxiv.org/abs/2310.03025
23. Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems, 20(4), 422–446. 10.1145/582415.582418
24. Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 150–158. 10.18653/v1/2024.eacl-demo.16
25. Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024), 194–199. 10.1145/3644815.3644945
26. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 79–90. 10.1145/3605764.3623985

27. Zou, W., Geng, R., Wang, B., & Jia, J. (2025). PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. Proceedings of the 34th USENIX Security Symposium, 3827–3844. www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag

28. Zeng, S., Zhang, J., He, P., Xing, Y., Liu, Y., Xu, H., Ren, J., Wang, S., Yin, D., Chang, Y., & Tang, J. (2024). The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG). arXiv preprint arXiv:2402.16893. arxiv.org/abs/2402.16893

### How to cite this article

Lee, Q. F. (2026). Retrieval-Augmented Generation: Design, Evaluation and Failure Analysis for Enterprise Knowledge Systems. *Chmltech Online Journal*, *104*(3). https://www.chmltech.com/post/retrieval-augmented-generation-design-evaluation-and-failure-analysis

## Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

**Book a Strategy Call**

Or email **info@chmltech.com**